CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment 6

Student Name: Manasvi Sharma **UID: 23BCS11815**
Branch: BE CSE **Section/Group: KRG 3A**
Semester: 6th **Date of Performance: 02/03/26**
Subject Name: Full Stack Development **Subject Code: 23CSH-309**

Aim:
To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

Objective:
After completing this experiment, students will be able to:

1. Understand Spring Boot fundamentals
   - Learn the role of Spring Boot in backend development.
   - Configure and run a basic Spring Boot application.

2. Apply IoC and Dependency Injection
   - Understand how Spring manages object creation and dependencies.
   - Use annotations like @Autowired, @Service, and @Repository.

3. Implement Layered Architecture
   - Structure the application using Controller, Service, and Repository layers.
   - Understand separation of concerns in backend design.

4. Develop RESTful APIs
   - Create CRUD operations using Spring Boot REST controllers.
   - Design APIs using proper HTTP methods (GET, POST, PUT, DELETE).

5. Use DTOs for API communication
   - Implement DTO mapping to separate internal models from API responses.

6. Implement Validation
   - Apply validation using Jakarta Validation annotations (@NotNull, @Email, etc.).

7. Handle Exceptions Globally
   - Implement global exception handling using @ControllerAdvice.

8. Build the HealthHub Backend Module
   - Develop backend services for patient/health record management.

Implementation/Code:
PatientController.java:

```
package com.example.healthhub.controller;

import com.example.healthhub.dto.PatientDTO;
import com.example.healthhub.entity.Patient;
import com.example.healthhub.service.PatientService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/patients")
public class PatientController {

    @Autowired
    private PatientService service;

    @PostMapping
    public Patient create(@Valid @RequestBody PatientDTO dto){
```

```
        Patient patient = new Patient();
        patient.setName(dto.getName());
        patient.setEmail(dto.getEmail());
        patient.setAge(dto.getAge());

        return service.save(patient);
    }

    @GetMapping
    public List<Patient> getAll(){
        return service.getAll();
    }

    @GetMapping("/{id}")
    public Patient getById(@PathVariable Long id){
        return service.getById(id);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id){
        service.delete(id);
    }
}
```

PatientRepository.java:

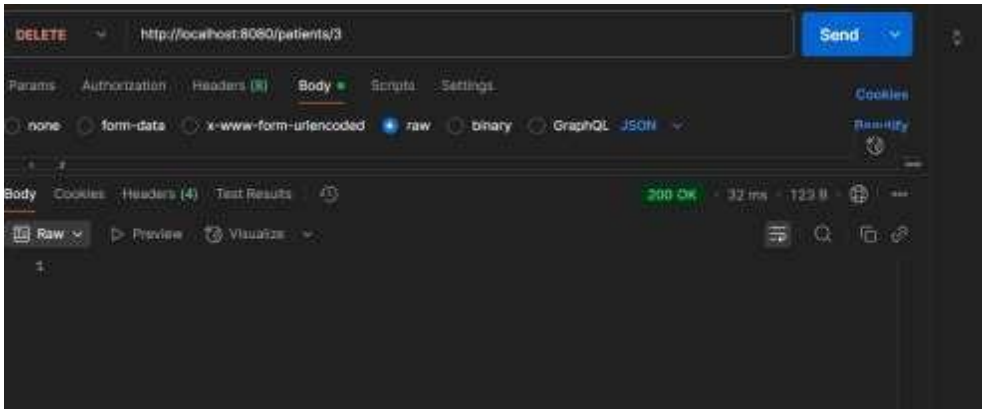
```
package com.example.healthhub.repository;

import com.example.healthhub.entity.Patient;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PatientRepository extends JpaRepository<Patient, Long> {
}
```

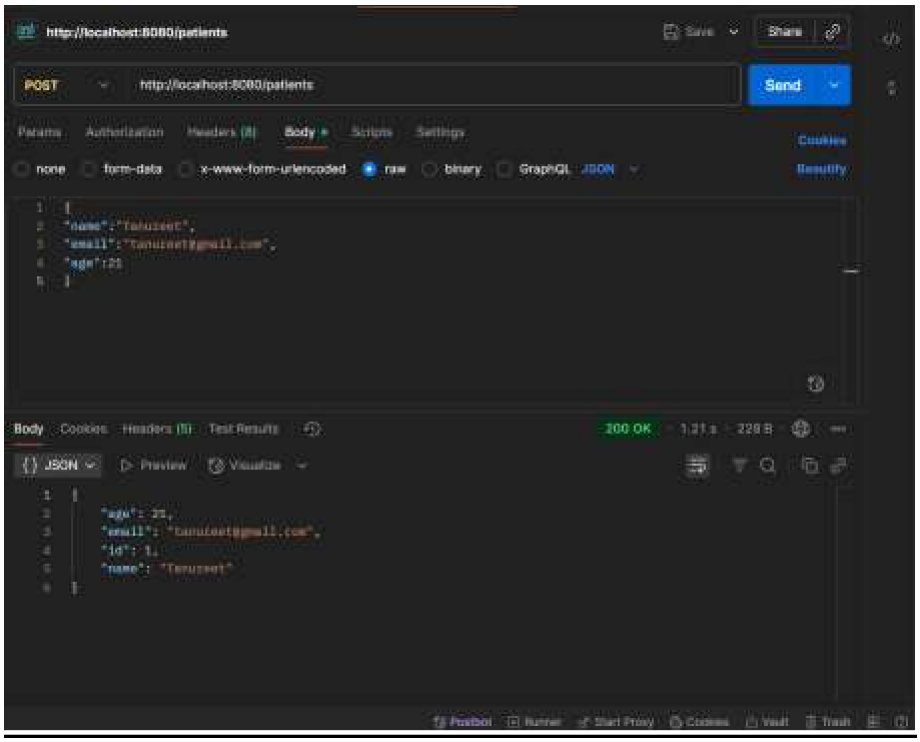
PatientService.java:

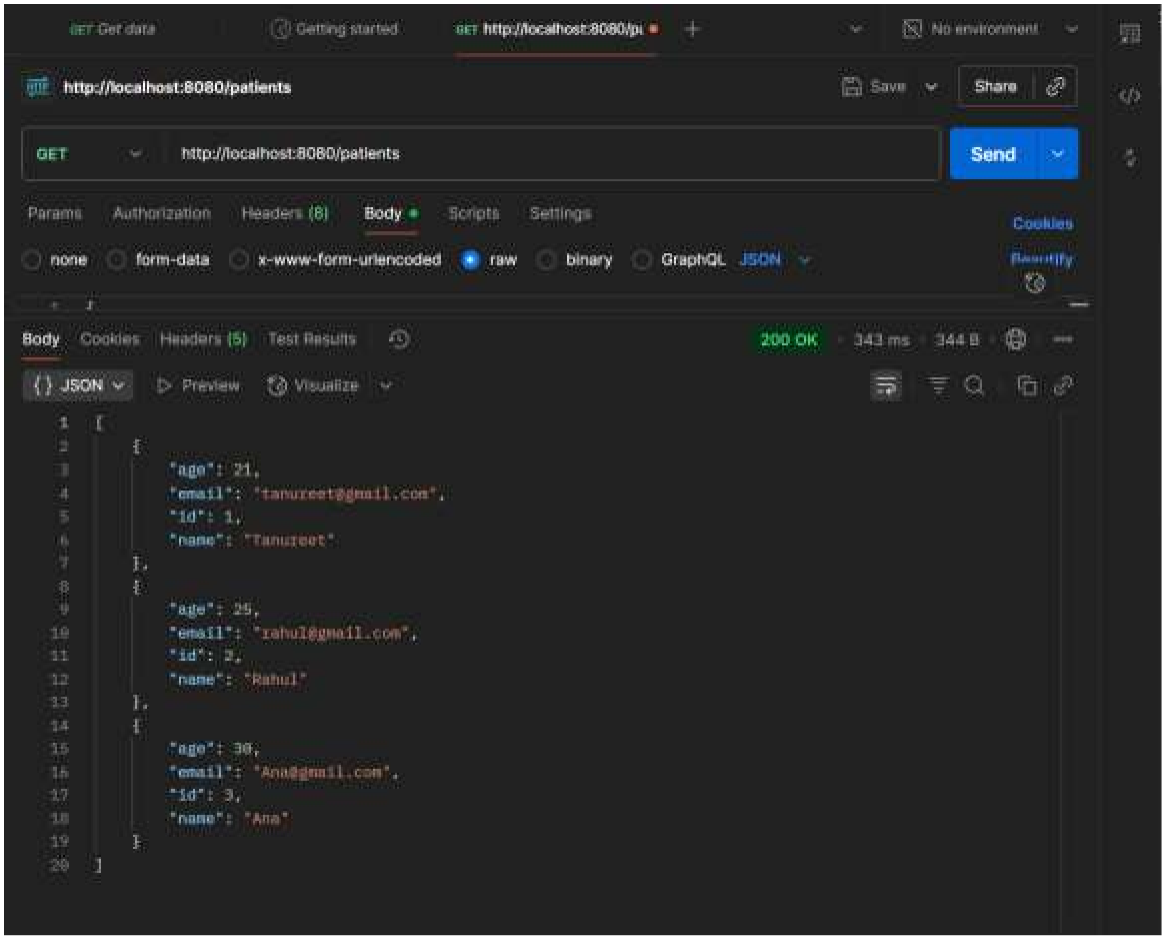
```
package com.example.healthhub.service;

import com.example.healthhub.entity.Patient;
import com.example.healthhub.repository.PatientRepository;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PatientService {

    @Autowired
    private PatientRepository repository;

    public Patient save(Patient patient){
        return repository.save(patient);
    }

    public List<Patient> getAll(){
        return repository.findAll();
    }

    public Patient getById(Long id){
        return repository.findById(id).orElse(null);
    }

    public void delete(Long id){
        repository.deleteById(id);
    }
}
```

Output:




http://localhost:8080/patients
POST
http://localhost:8080/patients
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"name":"Tanuzeet",
"email":"Tanuzeet@gmail.com",
"age":21
}
Body
Cookies
Headers (5)
Test Results
200 OK
1.21 s
228 B
JSON
Preview
Visualize
{
"age": 21,
"email": "tanuzeet@gmail.com",
"id": 1,
"name": "Tanuzeet"
}


GET Get data
Getting started
GET http://localhost:8080/pa
No environment
http://localhost:8080/patients
Save
Share
GET
http://localhost:8080/patients
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
343 ms
344 B
JSON
Preview
Visualize
[
{
"age": 21,
"email": "tanuzeet@gmail.com",
"id": 1,
"name": "Tanuzeet"
},
{
"age": 25,
"email": "rahul@gmail.com",
"id": 2,
"name": "Rahul"
},
{
"age": 30,
"email": "Ana@gmail.com",
"id": 3,
"name": "Ana"
}
]

GET Get data | Getting started | GET http://localhost:8080/pa | No environment

http://localhost:8080/patients/3 | Save | Share

GET http://localhost:8080/patients/3 | Send

Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Beautify

Body | Cookies | Headers (5) | Test Results | 200 OK · 33 ms · 218 B

JSON | Preview | Visualize


```
{
    "age": 30,
    "email": "Ana@gmail.com",
    "id": 3,
    "name": "Ana"
}
```


http://localhost:8080/patients | Save | Share

GET http://localhost:8080/patients | Send

Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Beautify

Body | Cookies | Headers (5) | Test Results | 200 OK · 28 ms · 289 B

JSON | Preview | Visualize


```
[
    {
        "age": 21,
        "email": "tanureet@gmail.com",
        "id": 1,
        "name": "Tanureet"
    },
    {
        "age": 25,
        "email": "rahul@gmail.com",
        "id": 2,
        "name": "Rahul"
    }
]
```

Learning Outcome:

- **Develop a Spring Boot application**
  Create and run a basic Spring Boot project and understand its role in modern backend development.
- **Apply Inversion of Control (IoC) and Dependency Injection (DI)**
  Implement IoC and DI concepts using Spring annotations such as @Autowired, @Service, and @Repository to manage dependencies efficiently.
- **Design applications using Layered Architecture**
  Structure backend applications into Controller, Service, and Repository layers, ensuring proper separation of concerns and maintainable code.
- **Create RESTful APIs with CRUD functionality**
  Develop REST APIs using appropriate HTTP methods (GET, POST, PUT, DELETE) to perform Create, Read, Update, and Delete operations.
- **Implement DTO-based data transfer**
  Use Data Transfer Objects (DTOs) to separate internal entity models from API request and response structures.